Universidad de San Carlos de Guatemala

Centro Universitario de Occidente CUNOC

División de las Ciencias de la Ingeniería

Estructura de Datos

Sección: A

Docente: Mario Moisés Ramírez Tobar

“Reporte Tecnico Estructura de Datos”

Estudiante:

Mynor Miguel Monzón Martínez 202230884

Quetzaltenango, 03 de abril de

2026

1. Diseño de las Estructuras

En esta sección explico cómo diseñé cada una de las estructuras de datos que implementé para el proyecto.

Clase Producto

Empecé definiendo una estructura simple llamada Producto que contiene todos los atributos que pide el enunciado: nombre, código de barras, categoría, fecha de vencimiento, marca, precio y stock. Usé un struct en lugar de una clase porque solo necesito que almacene datos, sin comportamientos complejos. El código de barras lo usé como identificador único, así que validé que no se repitan al momento de insertar.

Lista Enlazada (template)

Implementé una lista enlazada genérica usando templates. La lista tiene dos formas de inserción:

- insertarAlFinal(): para la lista no ordenada, simplemente agrega al final recorriendo toda la lista (O(n)).
- insertarOrdenadoPorNombre(): para la lista ordenada, inserta manteniendo el orden alfabético por nombre.

Decidí usar templates para poder reutilizar la misma lista con diferentes tipos de datos. La lista me sirvió como caso base para comparar el rendimiento de búsqueda secuencial contra el AVL.

Arbol AVL

El AVL lo implementé ordenado por nombre del producto. La razón es que el enunciado pedía búsqueda por nombre, y un árbol binario balanceado me da O(log n).

Lo que hice fue:

- Cada nodo guarda un producto y su factor de equilibrio (fe).
- Después de cada inserción o eliminación, verifico si el árbol está balanceado.
- Si no, aplico una de las cuatro rotaciones (II, DD, ID, DI).

Me costó un poco entender las rotaciones al principio, pero después de hacer varios ejemplos en papel logré implementarlas bien. La función balancear() revisa el factor de equilibrio y decide qué rotación aplicar.

Abol B (orden 2)

Para el árbol B usé orden d=2, lo que significa que cada nodo puede tener entre 2 y 4 llaves (o entre 1 y 2 si es hoja). La clave que usé fue la fecha de vencimiento.

Las partes más complicadas fueron:

- Split: cuando un nodo se llena, lo divido en dos y subo la llave mediana al padre.
- Eliminación: tuve que implementar préstamo entre hermanos y fusión de nodos.

Aunque el árbol B es más difícil de implementar que el AVL, la ventaja es que hace menos accesos a disco (aunque en este proyecto todo está en memoria, igual aprendí el concepto).

Arbol B+

El árbol B+ es similar al B, pero con una diferencia importante: todas las llaves están en las hojas. Los nodos internos solo guardan "rutas" para guiar la búsqueda. Además, las hojas están enlazadas entre sí mediante un puntero siguiente.

Usé la categoría como clave porque el enunciado pedía búsqueda por categoría. La ventaja del B+ es que cuando busco "todos los productos de una categoría", solo bajo hasta la primera hoja que contiene esa categoría y luego recorro las hojas usando el puntero siguiente. Esto es mucho más eficiente que recorrer todo el árbol.

Tabla Hash

La tabla hash la implementé con un tamaño inicial de 101 (elegí un número primo para mejorar la distribución). Para manejar colisiones usé encadenamiento: cada posición del arreglo contiene una lista enlazada de productos.

La función hash que usé es el algoritmo djb2, que fue creado por Dan Bernstein. Es bastante simple pero efectivo: recorre cada caracter del código de barras y va actualizando un valor hash con la fórmula hash = ((hash << 5) + hash) + c. Al final, tomo el módulo con el tamaño de la tabla.

Elegí encadenamiento en lugar de sondeo abierto porque es más fácil de implementar y no hay problema si la tabla se llena mucho, solo las listas se hacen más largas.

Pila para Rollback

Para implementar el rollback (deshacer eliminaciones) usé una pila. Cada vez que elimino un producto, guardo una copia en la pila. Si el usuario quiere deshacer, saco el producto de la pila y lo inserto de nuevo. La pila es perfecta para esto porque sigue el principio LIFO (último en entrar, primero en salir).

Clase Catalogo (Fachada)

Esta es la clase más importante porque une todo. El catálogo contiene una instancia de cada estructura de datos. Cuando el usuario agrega un producto, el catálogo lo inserta en todas las estructuras. Cuando elimina, lo elimina de todas. Así me aseguro de que los datos sean consistentes. Si algo falla durante la inserción, el método agregarProducto()retorna false y no se completa la operación. Para la eliminación, guardo una copia en la pila antes de eliminar, por si el usuario quiere hacer rollback.

2. Justificación de Big-O

En esta sección justifico la complejidad teórica de cada operación importante.

Lista Enlazada

Operación	Complejidad	Justificación
insertarAlFinal	O(n)	Tengo que recorrer toda la lista hasta encontrar el último nodo
insertarOrdenado	O(n)	Busco la posición correcta recorriendo los nodos
buscarPorCodigo	O(n)	En el peor caso, el producto está al final o no existe, y recorro todo
eliminarPorCodigo	O(n)	Primero busco el nodo (O(n)) y luego elimino (O(1))

Arbol AVL

Operación	Complejidad	Justificación
insertar	O(log n)	La altura del árbol está garantizada por el balanceo, así que en cada nivel hago una comparación
buscar	O(log n)	Misma razón: recorro la altura del árbol
eliminar	O(log n)	Busco el nodo (log n) y luego rebalanceo hacia arriba (log n)
rotaciones	O(1)	Cada rotación solo cambia unos pocos punteros

Arbol B (orden 2)

Operación	Complejidad	Justificación
insertar	O(log n)	

Operación	Complejidad	Justificación
		Recorro desde la raíz hasta la hoja, y en cada nivel hago split si es necesario
buscar	O(log n)	Cada nivel reduce el espacio de búsqueda
buscarPorRango	O(log n + k)	Bajo hasta la primera llave del rango (log n) y luego recorro (k)
eliminar	O(log n)	Busco, elimino y si hay underflow hago préstamo o fusión

Arbol B+

Operación	Complejidad	Justificación
insertar	O(log n)	Igual que el árbol B
buscarPorCategoria	O(log n + k)	Bajo a la hoja (log n) y luego recorro hojas enlazadas (k)

Tabla Hash

Operación	Complejidad	Justificación
insertar	O(1) promedio	Calculo el hash (O(k) donde k es longitud del código, pero se considera constante) y agrego a la lista
buscar	O(1) promedio	Misma razón
eliminar	O(1) promedio	Misma razón
Peor caso	O(n)	Si todos los productos caen en la misma cubeta, la lista se vuelve larga

Pila

Operación	Complejidad	Justificación
push	O(1)	Solo inserto al inicio
pop	O(1)	Solo elimino del inicio
peek	O(1)	Solo miro el tope

3. Metodología de Medición

Para medir el rendimiento implementé una clase llamada Benchmark. Esta clase usa std::chronopara medir tiempos con alta precisión (microsegundos).

Cómo funciona Benchmark

```
Benchmark timer;
timer.iniciar("Busquedapornombre");
Producto*p=catalogo.buscarPorNombre("Leche");timer.finalizar();
```

iniciar()guarda el tiempo actual. finalizar()calcula la diferencia y la muestra en consola.

Tipo de medición	**Qué estructuras comparé**	**Cómo lo hice**
Búsqueda por nombre	Lista ordenada vs AVL	Busqué 20 nombres aleatorios y promedié
Búsqueda por código	Tabla Hash	Busqué 20 códigos aleatorios
Búsqueda por categoría	Árbol B+	Busqué 3 categorías diferentes
Búsqueda por rango de fechas	Árbol B	Busqué rangos de 30 días
Inserción	Todas las estructuras	Medí el tiempo de agregar 100 productos

Escenario de prueba

- Productos cargados: 1000 (desde CSV)
- Consultas por tipo: 20 búsquedas exitosas + 5 fallidas
- Repeticiones: 5 veces por cada medición, luego promedié
- Unidad de medida: Microsegundos (µs)

Limitaciones

Sé que mis mediciones no son 100% precisas porque el sistema operativo puede interrumpir el programa en cualquier momento, pero al repetir varias veces y promediar, los resultados son baswtante confiables.

4. Resultados (Tablas y Plots)

Busqueda por Nombre: Lista vs AVL

Tipo de búsqueda	Lista Enlazada (µs)	Árbol AVL (µs)	Diferencia
Mejor caso (primero)	2	8	AVL más lento
Caso promedio	145	12	AVL es ~12x más rápido
Peor caso (último)	287	15	AVL es ~19x más rápido
Búsqueda fallida	301	16	AVL es ~18x más rápido

Busqueda por Codigo de Barras (Tabla Hash)

Tipo de búsqueda	Tiempo (µs)
Promedio exitoso	3
Promedio fallido	2

Factor de carga: 1000/101 ≈ 9.9 productos por cubeta

Busqueda por Categoria (Arbol B+)

Categoría	Productos encontrados	Tiempo (µs)
"Lácteos"	87	18
"Bebidas"	124	22
"Limpieza"	43	14

Promedio: 18 µs

Busqueda por Rango de Fechas (Arbol B)

Rango	Productos encontrados	Tiempo (µs)
2025-01-01 a 2025-06-30	312	45
2025-07-01 a 2025-12-31	288	42
2026-01-01 a 2026-03-31	156	28

Promedio: 38 µs

Insercion de Productos

Estructura	Tiempo por inserción (µs)	100 inserciones (µs)
Lista (no ordenada)	145	14,500
Lista ordenada	152	15,200
AVL	18	1,800
Árbol B	22	2,200
Árbol B+	24	2,400
Tabla Hash	3	300

Resumen General

Estructura	Mejor para...	Complejidad real
Lista Enlazada	(solo comparativa)	Lenta pero simple
AVL	Búsqueda por nombre	Muy rápida (log n)
Árbol B	Rango de fechas	Rápida, buena para rangos
Árbol B+	Búsqueda por categoría	Rápida, excelente para agrupar
Tabla Hash	Búsqueda por código	Súper rápida (O(1))
Pila	Rollback	Instantánea

5. Conclusiones

Sobre las estructuras implementadas:

Después de implementar y probar todas las estructuras, puedo concluir lo siguiente:

- La tabla hash es la reina de las búsquedas puntuales. Para buscar por código de barras (clave única), no hay nada más rápido. El tiempo promedio fue de solo 3 microsegundos.
- El AVL es excelente para búsquedas por nombre. Comparado con la lista enlazada, el AVL fue entre 12 y 19 veces más rápido. La diferencia se notará aún más si el catálogo crece a 10,000 o 100,000 productos.
- El árbol B+ es ideal para consultas por categoría. La combinación de búsqueda logarítmica más recorrido lineal por hojas enlazadas es muy poderosa. No tuve que implementar nada especial para recuperar todos los productos de una categoría.
- El árbol B es perfecto para rangos de fechas. Preguntar "qué productos caducan entre enero y marzo" es muy eficiente. Si tuviera que hacer esto con una lista, tendría que revisar producto por producto.
- La lista enlazada es la más lenta pero necesaria. El proyecto pedía comparar con búsqueda secuencial, así que la incluí. Sus tiempos O(n) se notan mucho cuando n=1000.
- El rollback con pila funciona perfecto. Es simple y eficiente: O(1) para guardar y O(1) para recuperar.

Sobre las mediciones

Las mediciones confirmaron lo que dice la teoría:

- O(log n) es mucho mejor que O(n) para valores grandes
- O(1) promedio de la tabla hash es increíblemente rápido
- Las constantes ocultas importan: el AVL fue un poco más lento que el árbol B en inserciones, pero la diferencia no es significativa